

Case Study Project: Library Management System

Complete End-to-End Project Requirement Document

This case study is designed as a **real-time software development project** where participants will perform:

- Requirement Analysis
 - Database Design
 - Database Normalization
 - SQL Development
 - Backend Development (C#)
 - Frontend Development (Console UI)
 - Reporting
 - Performance Optimization using Indexes
-

Project Title

Library Management System

Business Scenario

A college library currently manages books and member records manually using registers and Excel sheets.

As the number of books and students increases, managing the records has become difficult.

The college management has decided to develop a **Library Management System** to automate the complete process.

The system should allow librarians to:

- Manage Books
- Manage Categories
- Manage Authors
- Manage Members
- Issue Books
- Return Books

- Search Books
- Generate Reports

The application should be developed using:

Front End

- C# Console Application

Back End

- SQL Server Database

Concepts to Implement

- Normalization
 - Primary Keys
 - Foreign Keys
 - CRUD Operations
 - Joins
 - Subqueries
 - Stored Procedures
 - Indexing
 - Exception Handling
-

Phase 1: Requirement Gathering

Functional Requirements

Book Management

The librarian should be able to:

- Add New Book
 - Update Book Details
 - Delete Book
 - View All Books
 - Search Book by Name
 - Search Book by Category
-

Author Management

The librarian should be able to:

- Add Author
 - Update Author
 - Delete Author
 - View All Authors
-

Category Management

The librarian should be able to:

- Add Category
 - Update Category
 - Delete Category
 - View Categories
-

Member Management

The librarian should be able to:

- Register Member
 - Update Member Details
 - Delete Member
 - View Members
-

Transaction Management

The librarian should be able to:

- Issue Book
 - Return Book
 - View Issued Books
 - View Returned Books
-

Reporting Module

The system should generate:

- Total Books
- Total Members
- Available Books
- Issued Books
- Overdue Books

- Most Borrowed Books
-

Phase 2: Database Design Challenge

Step 1

Create an Un-Normalized Table

LibraryID BookName Authors Categories MemberName IssueDate

Task 1

Identify the problems in the above structure.

Questions:

1. Why is this table not normalized?
 2. What data redundancy exists?
 3. What update anomalies exist?
 4. What insertion anomalies exist?
 5. What deletion anomalies exist?
-

Step 2

Convert the database into:

1NF

Create atomic columns.

2NF

Remove partial dependency.

Create separate tables:

- Books
- Authors
- Categories

3NF

Remove transitive dependency.

Create final tables:

- Books
- Authors
- Categories
- Members
- Transactions

Deliverable

Draw ER Diagram.

Include:

- Primary Keys
- Foreign Keys
- Relationships

Phase 3: Database Development

Create the following tables.

Authors

Fields:

AuthorID
AuthorName

Categories

Fields:

CategoryID
CategoryName

Books

Fields:

BookID
Title
ISBN
PublishedYear
Quantity
CategoryID
AuthorID

Members

Fields:

MemberID
MemberName
Email
Phone
Address

Transactions

Fields:

TransactionID
BookID
MemberID
IssueDate
DueDate
ReturnDate
Status

Task

Write SQL scripts for:

- CREATE TABLE
 - PRIMARY KEY
 - FOREIGN KEY
 - NOT NULL Constraints
-

Phase 4: SQL CRUD Operations

Task 1

Insert Data

Create SQL queries for:

- Add Book
- Add Author
- Add Category
- Add Member

Task 2

Update Data

Create SQL queries for:

- Update Book Quantity
- Update Member Phone Number

Task 3

Delete Data

Create SQL queries for:

- Delete Book
- Delete Member

Task 4

Read Data

Create SQL queries for:

- View All Books
 - View All Members
 - View Available Books
-

Phase 5: SQL Joins

Question 1

Display:

Book Name
Author Name
Category Name

Use JOIN.

Question 2

Display:

Book Name
Member Name
Issue Date

Use JOIN.

Question 3

Display all books and categories.

Use LEFT JOIN.

Question 4

Display members who have issued books.

Use INNER JOIN.

Phase 6: SQL Subqueries

Question 1

Find the most borrowed book.

Use Subquery.

Question 2

Find members who never borrowed books.

Use NOT IN Subquery.

Question 3

Find books that are never issued.

Use Subquery.

Question 4

Find members who borrowed more than 5 books.

Use Correlated Subquery.

Phase 7: Stored Procedures

Create Stored Procedures for:

Add Book

Input:

Title
ISBN
Year
Quantity

Add Member

Input:

Name
Email
Phone

Issue Book

Input:

BookID
MemberID

Return Book

Input:

TransactionID

Phase 8: Indexing

Assume:

100,000 Books
50,000 Members

Search performance is slow.

Task 1

Create Index on:

Book Title

Task 2

Create Index on:

Member Email

Task 3

Compare:

Before Index

After Index

Measure:

Execution Time
Logical Reads

Phase 9: Backend Development (C#)

Create a Console Application.

Create Models

Book.cs

Properties:

BookID
Title
ISBN
PublishedYear
Quantity

Member.cs

Properties:

MemberID
MemberName
Email
Phone

Transaction.cs

Properties:

TransactionID
BookID
MemberID
IssueDate
ReturnDate
Status

Create Repository Layer

Responsibilities

- Database Connection
 - Execute Queries
 - Execute Stored Procedures
-

Create Service Layer

Responsibilities

- Business Logic
 - Validation
 - Exception Handling
-

Phase 10: Front-End (Console UI)

Create a menu-driven application.

Main Menu

```
=====
      LIBRARY MANAGEMENT
=====
```

1. Book Management
 2. Author Management
 3. Category Management
 4. Member Management
 5. Issue Book
 6. Return Book
 7. Search Book
 8. Reports
 9. Exit
-

Book Management Menu

1. Add Book
 2. Update Book
 3. Delete Book
 4. View Books
-

Member Management Menu

1. Add Member
 2. Update Member
 3. Delete Member
 4. View Members
-

Phase 11: Validation Rules

Implement validation.

Book

- Title cannot be empty.
 - Quantity cannot be negative.
-

Member

- Email must be unique.
 - Phone number should contain 10 digits.
-

Issue Book

- Book must exist.
 - Quantity must be greater than 0.
-

Phase 12: Reports Module

Generate reports using SQL Joins and Subqueries.

Report 1

Total Books

Report 2

Total Members

Report 3

Issued Books

Report 4

Overdue Books

Report 5

Most Borrowed Books

Report 6

Available Books

Final Deliverables

Database

- ✓ Normalized Database Design (1NF, 2NF, 3NF)
 - ✓ ER Diagram
 - ✓ Tables
 - ✓ Constraints
 - ✓ Indexes
 - ✓ Stored Procedures
-

Backend

- ✓ Models
 - ✓ Repository Layer
 - ✓ Service Layer
 - ✓ SQL Connection
 - ✓ CRUD Operations
-

Frontend

- ✓ Console Menus
 - ✓ User Input Screens
 - ✓ Report Screens
 - ✓ Search Screens
-

Expected Outcome

At the end of this case study, participants should be able to:

- Design a normalized database.
- Create SQL Server tables and relationships.
- Write CRUD queries.
- Use Joins and Subqueries.
- Implement Indexing.
- Create Stored Procedures.
- Build a layered C# Console Application.
- Connect C# with SQL Server.
- Generate business reports from SQL data.

This is the kind of complete case study that can be given as a **3–5 day corporate training project, college mini-project, or SQL + C# assessment project.**